

ThermoTrack - Monitoramento Inteligente de Temperatura: Uma Solução Preditiva e Autônoma para Ambientes Industriais

ThermoTrack - Intelligent Temperature Monitoring: A Predictive and Autonomous Solution for Industrial Environments

Benafle Soares da Hora, Bruno Orati, João Pedro Ferreira, Victor Eduardo Almeida, Vitor Orati

Serviço Nacional de Aprendizagem Comercial – SENAC

Engenharia da Computação - Bacharelado em Engenharia

benafle.soares@gmail.com, bruno.orati.sena@gmail.com, pellegrinijoaopedro@gmail.com, v2.abispo@gmail.com, vitor.orati.sena@gmail.com

Resumo. Este artigo apresenta o desenvolvimento de um sistema de monitoramento térmico inteligente voltado para aplicações em ambientes industriais, o *ThermoTrack*. Utilizando o sensor infravermelho AMG8833 acoplado ao microcontrolador ESP32, o sistema realiza a coleta de dados térmicos em tempo real e transmite as informações a um servidor remoto por meio de uma rede Wi-Fi. Paralelamente, um segundo módulo ESP32 conectado ao shield industrial OPS010 atua como nó atuador, realizando decisões automáticas com base nos dados recebidos. O sistema é capaz de acionar sinalizadores, ventoinhas e LEDs indicadores, proporcionando uma resposta eficiente frente a variações térmicas críticas. Com interface local (display LCD) e controle remoto via dashboard, o projeto mostra-se como uma alternativa viável de baixo custo e alta aplicabilidade na manutenção preditiva de sistemas industriais, contribuindo para a segurança, a confiabilidade e a continuidade operacional de máquinas e equipamentos.

Palavras-chave: Monitoramento térmico, Internet das Coisas (IoT), manutenção preditiva, Indústria 4.0, automação.

Abstract. This article presents the development of an intelligent thermal monitoring system aimed at applications in industrial environments, *ThermoTrack*. Using the AMG8833 infrared sensor coupled to the ESP32 microcontroller, the system collects thermal data in real time and transmits the information to a remote server via a Wi-Fi network. In parallel, a second ESP32 module connected to the OPS010 industrial shield acts as an actuator node, making automatic decisions based on the data received. The system is capable of activating beacons, fans and indicator LEDs, providing an efficient response to critical thermal variations. With a local interface (LCD display) and remote control via dashboard, the project proves to be a viable low-cost and highly applicable alternative in the predictive maintenance of industrial systems, contributing to the safety, reliability and operational continuity of machines and equipment.

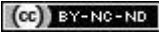
Key words: Thermal monitoring, Internet of Things (IoT), predictive maintenance, Industry 4.0, automation.

Contextos da Alimentação – Revista de Comportamento, Cultura e Sociedade

Vol. x no x – novembro de 2024, São Paulo: Centro Universitário Senac

ISSN 2238-4200

Portal da Revista Iniciação Senac: <https://www1.sp.senac.br/hotsites/blogs/revistainiciacao/> E-mail: divulgacaocientifica@sp.senac.br

Esta obra está licenciada com uma Licença [Creative Commons Atribuição-Não Comercial-SemDerivações 4.0 Internacional](https://creativecommons.org/licenses/by-nc-nd/4.0/) 

1. Introdução

Contextualização e Motivação

O presente artigo é resultado do Projeto Integrador desenvolvido no quinto semestre do curso de Bacharelado em Engenharia da Computação (BEC), no Centro Universitário Senac. Neste projeto representamos o grupo 4.

O trabalho foi resultado das aulas de PROJETO: SIMULAÇÃO COM MODELAGEM, OPERAÇÃO E GESTÃO I – PRÁTICAS DE EXTENSÃO, ministradas pelos professores Marcello Marcelino Correia e André Luiz, mas é também englobado em todas as outras matérias do curso da Engenharia para a tentativa de incorporar e praticar informações, matérias e aprendizados que compõem as disciplinas deste semestre: ESTRUTURAÇÃO DE DADOS e BANCO DE DADOS; dentro do espaço de acolhimento e integração da graduação presencial. Vale ressaltar que há disciplinas que não foram mencionadas por não possuírem muita relação com o projeto e outras que tiveram mais ênfase e aplicabilidade neste projeto, além das disciplinas aprendidas nos semestres anteriores do curso, que foram de suma importância para o desenvolvimento dos conhecimentos aplicados neste trabalho, a considerar que o curso é um processo de aprendizagem envolvendo os conhecimentos adquiridos, como automação, programação embarcada, eletrônica digital, sensores e redes de computadores.

A empresa **Opller**, especializada em soluções de automação e controle industrial, foi a referência técnica adotada no desenvolvimento do projeto, responsável por fornecer componentes desenvolvidos como o **Shield OPPLER OPS010**. Os alunos foram desafiados a propor uma solução realista e funcional que pudesse demonstrar a aplicabilidade de dispositivos embarcados em ambientes industriais, utilizando o Shield dado aos grupos. Assim, foi idealizado um sistema de **Monitoramento Térmico Inteligente**, com foco em **manutenção preditiva de equipamentos**.

O ponto de partida foi a constatação de que falhas térmicas — como superaquecimento de componentes, mau contato elétrico ou sobrecarga — são causas comuns de interrupções operacionais em ambientes industriais. Tais falhas, se não forem identificadas precocemente, podem gerar danos permanentes a máquinas, elevar custos de manutenção e comprometer a segurança do ambiente de trabalho.

O sistema desenvolvido é composto por dois módulos *ESP32* interligados via Wi-Fi. O primeiro módulo, com o *sensor térmico AMG8833*, coleta uma matriz de temperatura (8x8), exibe os dados localmente em um display e os envia para um servidor online. O segundo módulo, conectado ao Shield Industrial *OPS010*, consulta os dados recebidos e, com base em limites configurados, aciona automaticamente uma ventoinha, um sinaleiro e LEDs indicadores. A solução permite resposta térmica automática com visualização local e remota, utilizando tecnologias acessíveis e confiáveis. Ao sistema foi dado o nome de **ThermoTrack** visando um nome comercial, fácil de lembrar e memorizar e que tenha relação a função do projeto.

O sistema, por sua arquitetura modular, escalável e baseada em tecnologias abertas, se mostra uma solução viável para aplicação em fábricas, painéis elétricos, laboratórios ou qualquer outro ambiente onde a temperatura seja uma variável crítica. A proposta é compatível com os princípios da **Indústria 4.0**, ao combinar sensores, conectividade, inteligência e automação em um único projeto funcional.

Em semestres passados, o grupo já havia manuseado projetos da área de entretenimento, ambiente hospitalar e ambiente industrial, destacando-se um braço robótico autônomo e controlável para sistemas de produção industriais, e neste semestre conclui-se novamente a área de atuação industrial. Mas diferentemente dos anos passados, realizou-se pela primeira vez o uso do microcontrolador *ESP32*, muito mais eficaz e potente do que o *Arduino*.

Objetivos gerais

Com as devidas designações de tema e componentes obrigatórios, o objetivo principal deste projeto é o desenvolvimento de um sistema inteligente de monitoramento térmico, aplicável em ambientes industriais, capaz de identificar variações anormais de temperatura em tempo real e responder automaticamente a essas condições com ações corretivas. Essa proposta visa integrar tecnologias modernas e acessíveis de Internet das Coisas (IoT) com práticas de manutenção preditiva, promovendo maior segurança operacional, eficiência energética e redução de custos com falhas inesperadas.

Esse objetivo está diretamente alinhado com os princípios da Indústria 4.0, buscando soluções escaláveis e adaptáveis para o monitoramento de máquinas e equipamentos que, por natureza, estão sujeitos ao superaquecimento ou falhas térmicas, gerando assim o foco no ambiente industrial (critério de uso recomendado pelos professores e claramente a empresa Oppler).

Inclui-se a integração de sensores para monitoramento e controle de temperatura, o emprego de circuitos elétricos para funcionamento do sistema de análise e o sistema de ação, e programação avançada, bem como a aplicação de conceitos de automação industrial, qualidade e eficiência no processo.

É de grande interesse gerar a discussão crítica, a identificação de problemas, estudo das causas e propostas de soluções em grupo, nos variados produtos e/ou serviço da Oppler – nas etapas de estratégia, projeto, planejamento e controle, ou aprimoramento –, e, este objetivo reflete a integração dos conhecimentos adquiridos ao longo do curso de Engenharia da Computação e Engenharia de Produção, aplicando ciência e tecnologia para criar uma solução prática e eficiente para aprimorar a qualidade do produto final.

Objetivos específicos

Para atingir o objetivo geral de desenvolver um sistema inteligente de monitoramento térmico em tempo real, foram definidos os seguintes objetivos específicos:

- Integrar o sensor infravermelho AMG8833 a um microcontrolador ESP32 para capturar uma matriz térmica de 64 pontos;
- Estabelecer comunicação via Wi-Fi entre dois módulos ESP32, permitindo envio e recebimento de dados entre sensor e atuador;
- Armazenar os dados em um banco de dados online, possibilitando consultas remotas e histórico de temperatura;
- Exibir as informações de forma clara e acessível, por meio de um display local (LCD) e de um painel web dinâmico;
- Implementar um sistema de resposta automática baseado em faixas de temperatura, com acionamento de LEDs, ventoinha e sinalizador de 24V utilizando o shield industrial OPS010.

Estrutura do Artigo

Este artigo está organizado em nove seções principais. A **Introdução** apresenta o contexto da manutenção preditiva, a motivação para o desenvolvimento de um sistema inteligente de monitoramento térmico e os objetivos do projeto. Em seguida, a seção de **Objetivos** delimita as metas gerais e específicas da pesquisa.

A seção de **Fundamentação Teórica** explora os principais conceitos relacionados à Internet das Coisas (IoT), sensores térmicos, microcontroladores e manutenção preditiva. A **Metodologia** detalha a divisão do projeto em dois módulos independentes — sensor e atuador —, além das ferramentas e tecnologias utilizadas.

Na seção de **Desenvolvimento do Circuito e Programação**, são apresentados os esquemas eletrônicos, os códigos utilizados nos microcontroladores e a lógica de funcionamento adotada no sistema. A **Integração com Banco de Dados e Interface Web** explica como os dados são enviados, armazenados e visualizados remotamente por meio do dashboard.

A seção de **Resultados e Testes Experimentais** discute os testes realizados, os comportamentos observados e a resposta do sistema frente às variações de temperatura. Por fim, a **Conclusão** traz as principais contribuições do projeto e possíveis caminhos para futuras melhorias. As **Referências** reúnem os materiais bibliográficos e técnicos consultados durante o desenvolvimento.

2. Metodologia

Para o desenvolvimento do sistema de monitoramento térmico inteligente, foi adotada uma abordagem prática e iterativa, aliando pesquisa teórica, testes experimentais, programação embarcada, construção física dos circuitos e integração de sistemas. A metodologia aplicada se divide em quatro frentes principais: análise do problema, escolha e validação dos componentes, desenvolvimento dos módulos (sensor e atuador) e integração com banco de dados e dashboard web.

Análise do Problema e Definição da Arquitetura

O projeto partiu da identificação de um desafio recorrente em ambientes industriais: *o controle de temperatura de equipamentos em tempo real e de forma automatizada*, de forma que energia e tempo fossem poupados, idealizando a aplicação em um sistema real da indústria. Após discussões em grupo e com o apoio dos professores orientadores, definiu-se que a arquitetura ideal envolveria dois microcontroladores ESP32 operando de forma independente, mas conectados via Wi-Fi. A escolha de separar os módulos em um *nó sensor* e um *nó atuador* aumentou a escalabilidade, a confiabilidade do sistema e permitiu uma divisão clara de responsabilidades.

Escolha e Validação dos Componentes

Durante as etapas iniciais do projeto, foram pesquisadas diversas opções de sensores térmicos e ambientais que pudessem atender ao objetivo de capturar variações de temperatura em tempo real. Abaixo estão alguns dos sensores avaliados, bem como os motivos pelos quais foram descartados ou escolhidos. O sensor escolhido foi o sensor térmico infravermelho **AMG8833**, que será descrito na Seção abaixo *Materiais Utilizados*.

Desenvolvimento dos Módulos

O **nó sensor** foi programado na IDE Arduino com as bibliotecas Adafruit_AMG88xx, TFT_eSPI, ArduinoJson e WiFi.h. O código realiza a leitura contínua da matriz térmica, identifica a temperatura máxima, converte os dados em um mapa colorido interpolado e envia os valores via requisição HTTP (método POST) para um script PHP hospedado em um servidor remoto.

O **nó atuador**, por sua vez, foi configurado para realizar requisições HTTP (método GET) a cada 5 segundos a um endpoint que retorna os valores atuais da temperatura, além dos limites de segurança definidos pelo usuário. Após o parse do JSON retornado, o sistema compara os valores e aciona os elementos físicos conforme a lógica:

- **Abaixo de 35 °C:** LED azul ligado (temperatura baixa segura);
- **Entre 35 °C e 39 °C:** LED verde ligado (temperatura dentro do normal);
- **Acima de 39 °C:** LED vermelho, ventoinha e sinaleiro ativados (temperatura crítica).

Para a interface física local, foi utilizado um **display LCD 16x2** conectado ao atuador, que exibe em tempo real os dados de temperatura recebidos.

Tecnologias Utilizadas

O desenvolvimento foi feito na **IDE Arduino** com uso extensivo de bibliotecas da comunidade. Para visualizar e testar resistores e conexões, foram utilizados aplicativos móveis como “ElectroDroid” e simulações no **Tinkercad Circuits**. O servidor foi implementado com **PHP 8.1** em ambiente Linux, com banco de dados **MySQL**, enquanto o frontend da dashboard foi feito em **HTML, CSS e JavaScript**.

Para desenvolvimento da caixa utilizada para armazenar os dois circuitos, utilizou-se o software **SolidWorks**, e a máquina de corte a laser disponibilizada pela instituição Senac Nações Unidas (onde este grupo estuda). A utilização deste software foi crucial para garantir um modelo que fizesse os componentes encaixarem perfeitamente, e gerar uma beleza mínima para o projeto, com os circuitos não soltos.

Apoio e Validações

O projeto foi desenvolvido ao longo do semestre com apoio do professor André Luiz, principalmente em reuniões aos sábados. Foi durante essas sessões que se definiu, por exemplo, a melhor abordagem de comunicação entre os ESPs — inicialmente prevista via RS485 ou UART física (pinos RX e TX), mas substituída pela conexão Wi-Fi por simplificar a arquitetura e reduzir problemas de sincronização.

3. Materiais e Componentes utilizados

No *módulo sensor*, o componente mais importante é o sensor térmico infravermelho **AMG8833**, fabricado pela Panasonic, foi escolhido por sua capacidade de captar uma matriz de temperatura 8x8, cobrindo uma área considerável com boa resolução espacial.

Figura 1. Sensor Infravermelho AMG8833.



Fonte: OkyStar [2]

Seu funcionamento via protocolo I2C e compatibilidade com o ESP32 foram fatores determinantes. Inspiramo-nos em implementações como a do vídeo [Electronic Clinic \(2020\)](#) [3], onde o sensor é integrado com um display TFT para visualização local, o que influenciou diretamente nosso circuito de leitura térmica. Este e outros vídeos foram de crucial ajuda ao projeto, proporcionando conhecimento nas possibilidades dos componentes, a sua programação extensa realizada, pinagens corretas etc.

Agora quando falamos dos sensores rejeitados, começamos com o melhor deles, o MLX90640. É um sensor de imagem térmica com uma matriz de **32x24 pixels**, desenvolvido pela Melexis. Ele oferece uma resolução significativamente superior ao AMG8833, com campo de visão amplo e detecção precisa de padrões de calor. Além disso, fornece leitura contínua via I2C ou SPI, o que o torna ideal para aplicações de mapeamento térmico detalhado.

Figura 2. Sensor Infravermelho MLX90640.



Fonte: Pimorini [4]

Apesar de sua alta qualidade, o MLX90640 possui um **custo** bastante elevado, podendo ultrapassar R\$ 500,00 por unidade no mercado nacional. Isso inviabilizaria a proposta de criar um sistema acessível e replicável.

Já o DHT11 é um sensor digital amplamente utilizado em projetos básicos de automação. Ele fornece dados de **temperatura e umidade** com interface simples e baixo custo. Sua popularidade se dá principalmente pela facilidade de uso em microcontroladores como Arduino e ESP32.

O MLX90614 é um sensor infravermelho sem contato que mede a temperatura de superfícies com boa precisão, utilizando comunicação I2C.

Figura 3. Sensor Infravermelho MLX90614.



Fonte: SmartKits [5]

Apesar de eficiente, ele realiza a leitura de **apenas um ponto** por vez, o que o torna inadequado para aplicações que exigem **mapeamento térmico em área**, como no nosso projeto. Por isso, foi descartado em favor do AMG8833, que possui matriz 8x8.

Figura 5. Sensor térmico e de umidade DHT11.



Fonte: SmartKits [6]

O DHT11 **não possui precisão nem velocidade** de resposta suficientes para aplicações industriais críticas. Além disso, ele fornece apenas a **temperatura ambiente em um ponto fixo**, o que limita drasticamente a detecção de pontos quentes em equipamentos.

Passando para o processamento e comunicação, utilizamos a placa **ESP32 DevKit v1**, por sua robustez, suporte à comunicação Wi-Fi integrada e disponibilidade de GPIOs.

Figura 5. ESP32 DevKit v1.



Fonte: Loja Vida de Silício [7]

Como meio de exibição local no módulo sensor, utilizamos um **display TFT SPI ILI9341**, com implementação de interpolação gráfica para simular um mapa térmico mais fluido processo feito com base em tutoriais da comunidade [Arduino e YouTube](#) [8].

Figura 6. Display TFT SPI ILI9341.



Fonte: Amazon [9]

No *módulo atuador*, diversos componentes foram incorporados para garantir a atuação eficiente e o monitoramento em tempo real.

Utilizamos outro ESP32 acoplado ao **Shield Industrial OPS010**, fornecido pela empresa **Oppler**, para controlar dispositivos de 24V com segurança. Este shield possui saídas digitais isoladas (Y1 a Y8), compatíveis com ventoinhas, sinalizadores e outros atuadores industriais (de 24V).

Figura 7. Shield Industrial OPS010.



Fonte: Oppler [10]

Vale lembrar que este aparelho foi o providenciado pela empresa Oppler, e aquele que foi designado como peça-chave para realizar os processos de atuação industrial em todos os projetos desenvolvidos pela turma do quinto semestre.

Um display LCD 16x2 foi integrado ao ESP32 com o objetivo de exibir localmente informações como temperatura atual, limites de segurança e o status do sistema. A visualização alfanumérica direta permite que o operador identifique rapidamente a condição térmica dos equipamentos. Para controle do contraste do display, foi utilizado um potenciômetro de 10k Ω , conectado entre VCC, GND e o pino VO do display.

Figura 8. Display LCD 16x2.



Fonte: UsinaInfo – Eletrônica e Robótica [11]

Como mecanismo de alerta visual e sonoro em situações críticas, foi adotado o **sinalizador BZ20-2L-R da Metaltex**, alimentado por 24V e conectado ao shield OPS010. Sua ativação automática informa imediatamente a ocorrência de temperaturas acima do limite de segurança estabelecido. Complementando essa função, um micro ventilador 24V foi instalado para atuar como elemento de resfriamento ativo, sendo acionado também via saída digital do shield industrial.

Figura 9. Sinalizador BZ20-2L-R.



Fonte: Sartori Casa & Construção [12]

O sistema utiliza ainda três LEDs indicadores de status: azul, verde e vermelho. Cada cor representa uma condição distinta — temperatura abaixo do mínimo, dentro da faixa ideal e acima do limite seguro, respectivamente. Todos os LEDs foram ligados com resistores de 220 Ω para garantir a proteção dos pinos do microcontrolador e limitar a corrente elétrica.

Figura 10. LED'S.



Fonte: Casa da Robótica [13]

Complementando o sistema de resposta automática, foi incorporado ao projeto um **micro ventilador 24V**, responsável pelo resfriamento ativo do ambiente monitorado. Conectado ao shield OPS010, o ventilador é acionado automaticamente sempre que a temperatura excede o limite crítico configurado no sistema, ajudando a reduzir o calor local e evitar danos aos equipamentos.

Figura 11. Micro Ventilador 24V.



Fonte: Boto Brasil [14]

A escolha por um modelo de 120 mm garante boa vazão de ar e baixo ruído, sendo ideal para uso em aplicações industriais e projetos embarcados. O ventilador opera com alimentação direta de 24V e foi fixado internamente na caixa do módulo atuador para maximizar a eficiência da ventilação.

A alimentação do circuito de atuação é feita por uma bateria de 24V, que fornece energia suficiente para os dispositivos industriais conectados, como o ventilador e o sinalizador. Para converter essa tensão em níveis compatíveis com o ESP32 (3.3V ou 5V), foi incorporado ao sistema um módulo regulador LM2596 de 3A. Esse conversor buck é responsável por reduzir a tensão de forma eficiente, assegurando a estabilidade da alimentação dos componentes sensíveis.

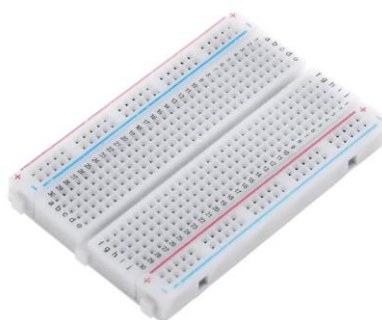
Figura 12. Fonte chaveada 24V bivolt.



Fonte: Loja Lunicon [15]

Durante o processo de montagem e testes, foi utilizada uma protoboard padrão de 830 pontos, o que facilitou ajustes rápidos e substituições de componentes sem necessidade de solda inicial. Contudo, para garantir firmeza nas conexões definitivas, diversos fios foram posteriormente soldados manualmente, principalmente nos pinos do display LCD e do sensor térmico AMG8833.

Figura 13. Protoboard.



Fonte: Casa da Robótica [16]

4. Desenvolvimento do projeto

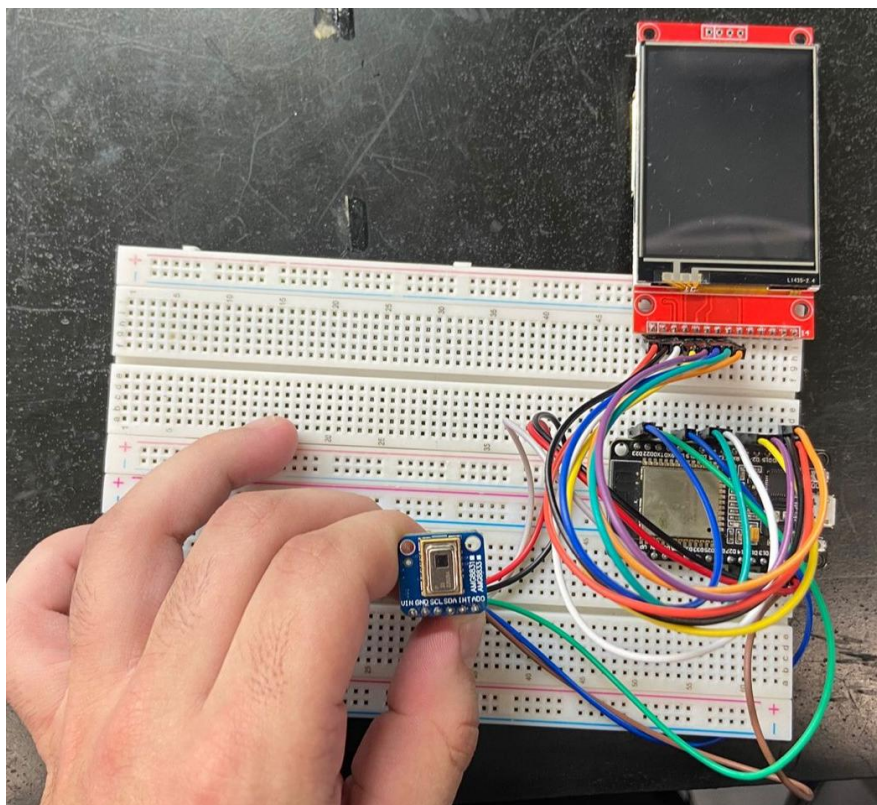
Montagem do Circuito

A montagem do circuito foi uma das etapas mais trabalhosas e determinantes para o funcionamento do sistema de monitoramento térmico inteligente. Inicialmente, os componentes foram organizados e testados individualmente em protoboards, garantindo a leitura dos sensores, acionamento dos atuadores e integridade dos sinais.

A Figura 13 apresenta o **módulo sensor**, montado sobre uma protoboard. Este circuito é composto por um microcontrolador ESP32 DevKit V1, responsável pela leitura, processamento e envio dos dados; o sensor térmico AMG8833, responsável pela captação da temperatura em tempo real; e um display TFT SPI (ILI9341), utilizado para visualização local da imagem térmica. O sensor foi conectado ao ESP32 via protocolo I2C, utilizando os pinos GPIO21 (SDA) e GPIO22 (SCL). A alimentação foi feita com 3.3V, com uso de resistores de pull-up para garantir a integridade do sinal. O display, por sua vez, foi conectado via protocolo SPI, utilizando os pinos GPIO23 (MOSI), GPIO18 (SCK), GPIO5 (CS), GPIO2 (DC) e GPIO4 (RESET), sendo alimentado com **5V**.

Na prática, o sensor AMG8833 é posicionado voltado para o ambiente ou equipamento alvo, detectando uma matriz de 64 pontos de temperatura (8x8). Os dados são processados no ESP32, interpolados para formar uma imagem térmica mais fluida e enviados para o display, que exibe as informações visuais juntamente com a **temperatura máxima detectada**. Simultaneamente, esses dados são armazenados e enviados via Wi-Fi para um **servidor remoto**, como será detalhado na seção sobre banco de dados.

Figura 14. Circuito do módulo sensor.



Fonte: Autoria própria.

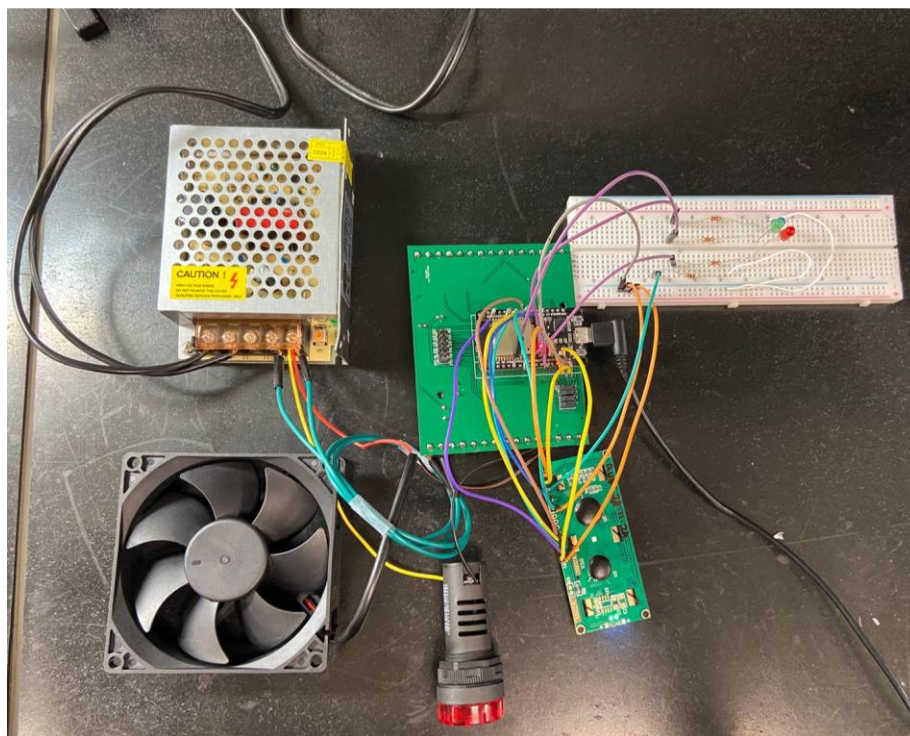
A Figura 14 mostra o **módulo atuador**, responsável por reagir às informações térmicas recebidas. Este circuito utiliza um segundo ESP32, conectado ao Shield Industrial OPS010, permitindo o acionamento de dispositivos de 24V com segurança, como uma ventoinha, um sinalizador sonoro e LEDs de indicação de estado. Também foi conectado um display LCD 16x2, controlado via interface paralela e acompanhado por um potenciômetro de 10k Ω , utilizado para ajuste de contraste da tela.

Neste módulo, o ESP32 consulta periodicamente o banco de dados (via Wi-Fi) e obtém os valores da temperatura máxima captada pelo módulo sensor, além dos **limites mínimo e máximo** definidos. De acordo com esses dados:

- Se a temperatura for **menor que 35 °C**, o LED **azul** é aceso (faixa segura);
- Se estiver **entre 35 °C e 39 °C**, o LED **verde** indica faixa ideal;
- Se ultrapassar **39 °C**, são acionados o **LED vermelho**, a **ventoinha de resfriamento** e o **sinalizador sonoro**, indicando situação crítica.

Essas respostas foram programadas previamente e permitem ação automatizada imediata em caso de risco térmico.

Figura 15. Circuito do módulo atuação.



Fonte: Autoria própria.

Durante o processo, diversos desafios foram enfrentados, como a instabilidade de conexões, necessidade de ressoldagem em pinos críticos e adaptações nos jumpers para encaixe em bornes industriais. Foi necessário o uso de resistores de 220Ω para proteção dos LEDs. Colocou-se um regulador LM2596 para reduzir a tensão da fonte de 24V para os níveis adequados as componentes como os led's e o display, que trabalham com 5V e/ou 3.3V. O esp32, pelo fato de estar acoplado com o Oppler, não precisava de regulação de tensão. Mas como já tinham que ser soldados as portas de comunicação digital do display LCD e dos LEDs, os 5 volts vieram diretamente da porta VCC e GND do próprio esp32 acoplado com o Oppler, evitando o uso do regulador de tensão LM2596.

A alimentação principal do circuito foi feita por uma fonte de 24V, alimentando o Shield OPS010 diretamente e, indiretamente, o ESP32 via regulador do próprio OPS. Essa configuração garantiu um fornecimento energético estável e seguro para todo o sistema.

A conclusão desta etapa marcou a validação elétrica completa do projeto, garantindo que os sinais estavam fluindo corretamente entre sensores, microcontroladores, atuadores e interfaces de exibição.

Programação

Nesta seção, iremos focar nas principais parte da programação, tanto do "módulo sensor" e do "módulo atuador".

O **módulo sensor** foi programado utilizando a IDE Arduino, com foco em captar a imagem térmica da matriz 8×8 do sensor AMG8833, interpolar os dados para gerar uma visualização mais suave e enviar as informações a um servidor via Wi-Fi.

Figura 16. Trecho com as bibliotecas e definições dos pinos TFT_DC, TFT_CS, TFT_RST.

```
#include "DFRobot_GDL.h"
#include <Wire.h>
#include <Adafruit_AMG88xx.h>

// Pinos do display
#define TFT_DC 2
#define TFT_CS 5
#define TFT_RST 4

DFRobot_ST7789_240x320_HW_SPI tft(TFT_DC, TFT_CS, TFT_RST);
```

Fonte: Autoria própria.

Logo após, o código inicializa o display TFT (modelo ILI9341 ou compatível com ST7789) via SPI, e configura o sensor AMG8833 via I2C. Os arrays pixels[] armazenam os valores de temperatura captados:

Figura 17. Trecho com definição dos arrays pixels[], pixels2[], e interpolação.

```
// Sensor térmico
Adafruit_AMG88xx amg;
#define AMG_COLS 8
#define AMG_ROWS 8
float pixels[AMG_COLS * AMG_ROWS];
float pixels2[AMG_COLS * AMG_ROWS];

// Tamanho da imagem interpolada
#define INTERPOLATED_COLS 30 /* Em 35 ele diminui a tela!! */
#define INTERPOLATED_ROWS 30 /* Em 35 ele diminui a tela!! */
```

Fonte: Autoria própria.

Também são definidos os tamanhos da imagem interpolada (30x30), para suavizar a visualização térmica.

O código define manualmente uma paleta de cores RGB565 (camColors[]) que será usada para representar os valores de temperatura com cores térmicas. Isso é essencial para a visualização correta no display.

[illegible]

Após iniciar o sensor e o display, o ESP32 conecta-se à rede Wi-Fi definida nas variáveis ssid e password. Em seguida, coleta as temperaturas e envia um JSON via HTTP POST para um servidor remoto.

- Temperatura máxima
- Temperatura mínima
- Temperatura média
- Temperaturas dos 64 pontos da matriz

```

/* BANCO DE DADOS */
void enviarDadosParaServidor(float maxTemp, float *matriz, int linhas, int colunas) {
    if (WiFi.status() == WL_CONNECTED) {
        HTTPClient http;
        http.begin("https://thermotrack.ganhemaishoje.com.br/insereDados.php");
        http.addHeader("Content-Type", "application/json");

        // Monta JSON com temperatura, dispositivo_id e imagem_base64
        String json = "{\"dispositivo_id\":\"1\"";
        json += ", \"temperatura_max\": " + String(maxTemp, 2);
        json += ", \"imagem_base64\": \"data:image/png;base64,\" + imagem_base64_fake + "\"}";

        int httpResponseCode = http.POST(json);
        Serial.print("Código resposta: ");
        Serial.println(httpResponseCode);
        Serial.println("Resposta: " + http.getString());

        http.end();
    }
}

```

Revista Iniciação - Vol. x no x – junho de 2025

O sensor capta 64 valores e, com auxílio de uma função de interpolação bicúbica, o código gera uma imagem térmica com transições suaves, exibida em tempo real no display.

Figura 20. Trecho com função de interpolação e renderização no display (`tft.fillRect()`) e função `drawpixels()`.

```
void drawpixels(float *p, uint8_t rows, uint8_t cols, uint8_t boxW, uint8_t boxH, boolean showVal) {
    // percorre todos os pontos interpolados
    for (int y = 0; y < rows; y++) {
        for (int x = 0; x < cols; x++) {
            float val = get_point(p, rows, cols, x, y);

            // mapeia val ∈ [MINTEMP, MAXTEMP] para idx ∈ [0, maxIndex]
            int idx = map(val, MINTEMP, MAXTEMP, 0, 239); // Sem multiplicar por 10
            idx = constrain(idx, 0, 239);

            // **AQUI**: só este fillRect usa camColors[idx]
            tft.fillRect(x * boxW, y * boxH, boxW, boxH, camColors[idx]);

            // rastreia o máximo para desenhar o cruz
            if (val > pix_max) {
                pix_max = val;
                pos_x = x * boxW;
                pos_y = y * boxH;
            }

            // (opcional) sobrepõe valor numérico
            if (showVal) {
                tft.setCursor(x * boxW + boxW / 2 - 12, y * boxH + boxH / 2 + 4);
                tft.setTextColor(COLOR_RGB565_WHITE);
                tft.setTextSize(1);
                tft.print(val, 1);
            }
        }
    }
}
```

Fonte: Autoria própria.

A função `drawpixels()` percorre todos os pontos interpolados da matriz térmica (64 valores), mapeando os valores de temperatura para uma escala de cores RGB565 contida no vetor `camColors[]`. Cada valor é desenhado no display como um retângulo colorido com `tft.fillRect()`, compondo a imagem térmica em tempo real. A função também rastreia o ponto de maior temperatura e, opcionalmente, sobrepõe os valores numéricos nos quadrantes da tela.

Em resumo, o sistema funciona de forma contínua:

- Lê os dados do sensor
- Atualiza o display com a imagem térmica interpolada
- Envia os dados para o banco de dados

Isso transforma o ESP32 com AMG8833 em uma câmera térmica portátil com interface visual local e capacidade de comunicação remota.

Agora quanto ao **módulo atuador**, o objetivo é consultar um banco de dados online com a última temperatura registrada pelo sensor, comparar essa temperatura com os limites definidos e acionar ventoinha, sinalizador sonoro e LEDs conforme a faixa da temperatura.

A primeira etapa da programação consiste em definir as bibliotecas utilizadas, para que haja funcionamento dos códigos que exigem tais bibliotecas específicas, como *WiFi.h*, *WiFiClientSecure.h*, *HTTPClient.h*, *ArduinoJson.h* e *LiquidCrystal.h*.

Era necessário conectar o ESP32 à rede Wi-Fi, para permitir que ele acesse o servidor remoto onde os dados de temperatura estão armazenados. Além disso, um display LCD 16x2 é inicializado para exibir localmente a temperatura e os limites definidos.

Figura 21. Código de inicialização da conexão Wi-Fi e do display LCD no módulo atuador.

```
const char *ssid = "iPhone de Vitor";
const char *password = "vitor.sena";
const char* host = "thermotrack.ganhemaishoje.com.br";

void setup() {
    Serial.begin(115200);
    pinMode(sinaleiro, OUTPUT);
    pinMode(ventoinha, OUTPUT);
    pinMode(ledAzul, OUTPUT);
    pinMode(ledVerde, OUTPUT);
    pinMode(ledVermelho, OUTPUT);

    lcd.begin(16, 2);
    lcd.print("Conectando...");

    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }

    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("WiFi conectado");
    Serial.println("\n[INFO] Conectado ao Wi-Fi");
    delay(1000);
}
```

Fonte: Autoria própria.

Com a conexão estabelecida, o ESP32 realiza periodicamente uma requisição HTTPS a um script PHP hospedado na web. Esse script retorna um JSON com a última temperatura registrada, além dos limites mínimo e máximo definidos pelo usuário via dashboard.

Figura 22. Requisição HTTPS e leitura do JSON retornado contendo os dados de temperatura.

```
void loop() {
    if (WiFi.status() == WL_CONNECTED) {
        WiFiClientSecure client;
        client.setInsecure(); // Desativa verificação do certificado SSL

        HTTPClient https;
        if (https.begin(client, "https://thermotrack.ganhemaishoje.com.br/statusatuais.php")) {
            int httpCode = https.GET();

            if (httpCode == 200) {
                String payload = https.getString();
                Serial.println("[INFO] Resposta recebida: " + payload);

                StaticJsonDocument<256> doc;
                DeserializationError err = deserializeJson(doc, payload);
            }
        }
    }
}
```

Fonte: Autoria própria.

Com os dados em mãos, o ESP32 executa a lógica de controle. Três faixas de temperatura foram definidas para orientar o acionamento dos dispositivos:

- Abaixo de 35 °C: LED azul indica temperatura segura.
- Entre 35 °C e 39 °C: LED verde indica faixa ideal.
- Acima de 39 °C: LED vermelho, ventoinha e sinalizador são acionados.

Figura 23. Trecho do código responsável pela tomada de decisão automática e acionamento dos atuadores.

```
digitalWrite(sinaleiro, LOW);
digitalWrite(ventoinha, LOW);
digitalWrite(ledAzul, LOW);
digitalWrite(ledVerde, LOW);
digitalWrite(ledVermelho, LOW);

if (temperatura < 35.0) {
    digitalWrite(ledAzul, HIGH);
    Serial.println("[ATUADOR] LED AZUL (abaixo)");
} else if (temperatura <= 39.0) {
    digitalWrite(ledVerde, HIGH);
    Serial.println("[ATUADOR] LED VERDE (ideal)");
} else {
    digitalWrite(ledVermelho, HIGH);
    digitalWrite(ventoinha, HIGH);
    digitalWrite(sinaleiro, HIGH);
    Serial.println("[ATUADOR] VERMELHO + VENTO + BUZINA (acima)");
}
```

Fonte: Autoria própria.

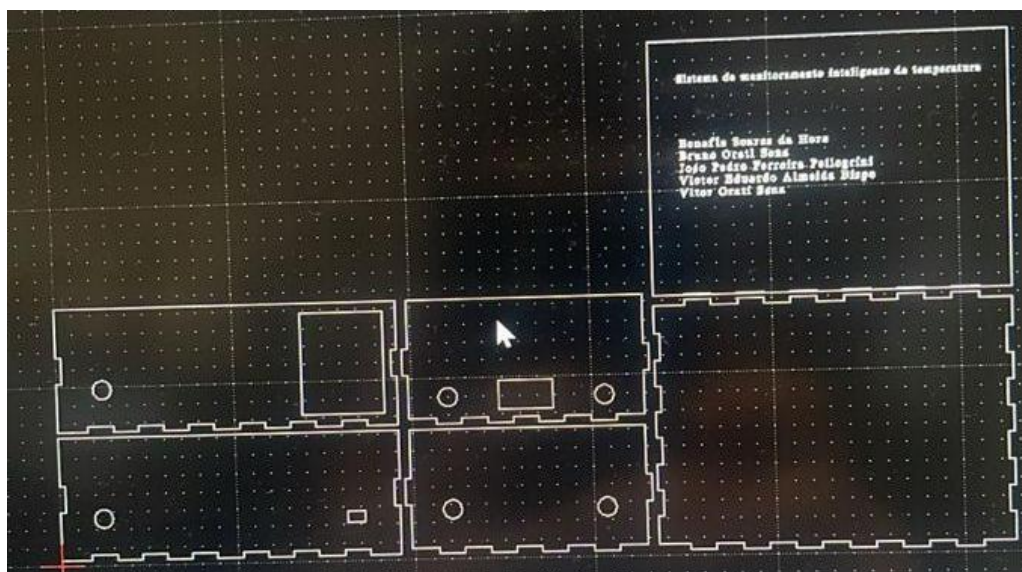
A figura 22 demonstra o trecho mais importante para explicar a **lógica de decisão** (ilustrar as faixas com base nas cores e atuadores). Quando for identificado uma temperatura maior que 39 (que é possível alterar de acordo com o uso do projeto), o LED vermelho, a ventoinha, e o sinalizador (buzzer) são acionados.

Modelagem e Corte na caixa de Proteção

Para garantir a organização e proteção dos circuitos, desenvolvemos uma caixa personalizada utilizando software de CAD. O desenho foi feito em ambiente 2D no LibreCAD, com todos os encaixes e furos planejados para os componentes internos, como o display, ESP32, sensor AMG8833, ventilador e sinalizador. Ademais, o SolidWorks teve um papel importante também para corrigir os erros que o LibreCAD não dava conta, além da familiaridade com o software.

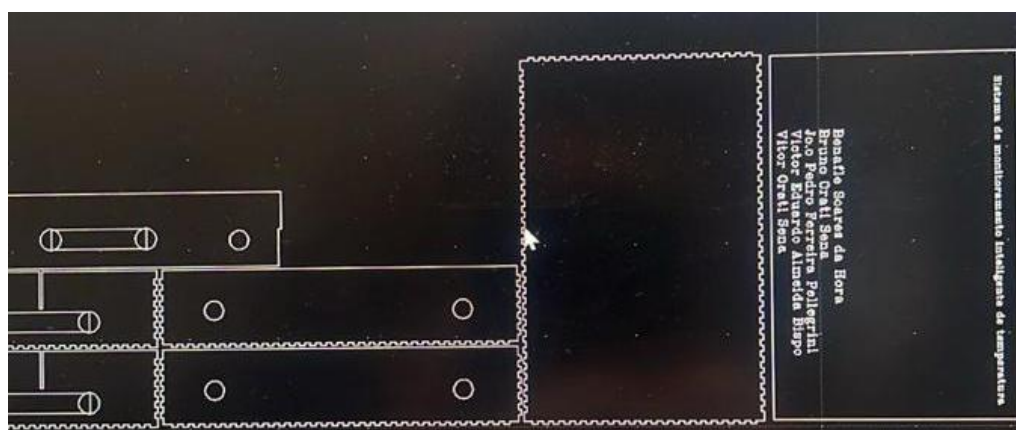
As figuras abaixo mostram o layout projetado, incluindo o posicionamento das faces, os encaixes e a gravação a laser do título do projeto com os nomes dos integrantes. Após validação no ambiente digital, o modelo foi exportado em formato .DXF e cortado a laser em acrílico 3 mm na estrutura do Senac, na máquina CNC (Controle Numérico Computadorizado, uma tecnologia que utiliza um computador para controlar e programar máquinas-ferramentas, incluindo máquinas de corte a laser).

Figura 24. Layout da caixa protetora no software CAD do módulo sensor.



Fonte: Autoria própria.

Figura 25. Layout da caixa protetora no software CAD do módulo atuador.



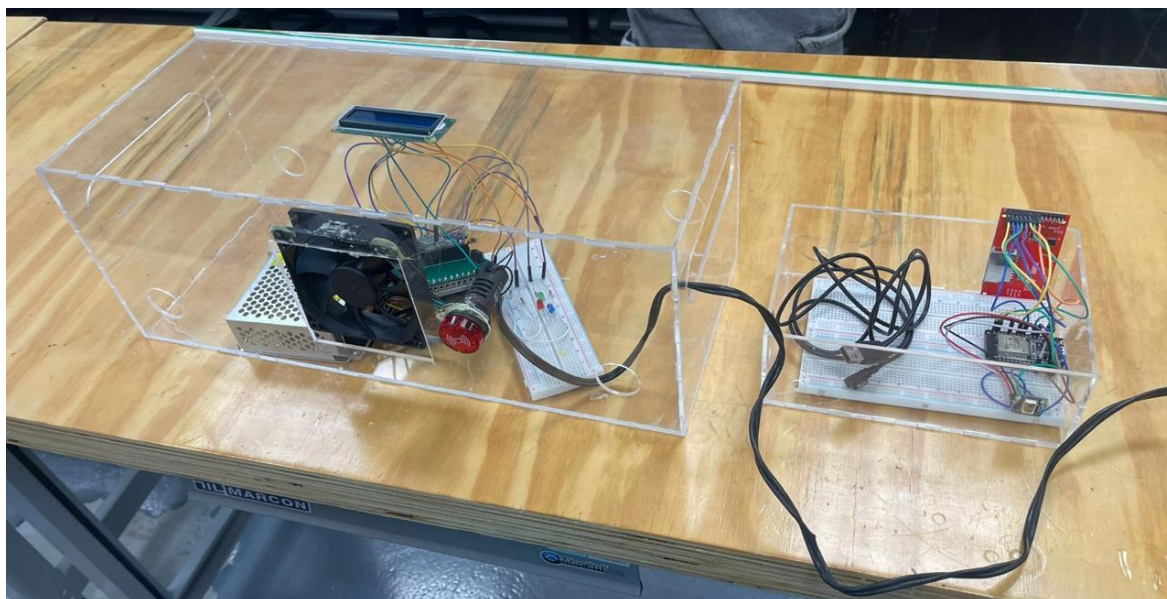
Fonte: Autoria própria.

A criação dessa estrutura foi fundamental para dar acabamento ao projeto e permitir transporte seguro, sem comprometer o funcionamento dos módulos eletrônicos internos.

Resultados

A Figura 25 apresenta o sistema completo já montado e organizado em duas caixas de acrílico desenvolvidas especialmente para acomodar os módulos sensor e atuador. À esquerda, encontra-se o módulo atuador, responsável pelo acionamento da ventoinha, sinaleiro e LEDs, enquanto à direita está o módulo sensor, que realiza a leitura térmica em tempo real com o sensor AMG8833 e exibe os dados em um display gráfico.

Figura 26. Sistema final montado em caixas de acrílico para os dois módulos.



Fonte: Autoria própria.

A montagem exigiu diversas etapas de adaptação e correções, especialmente durante a soldagem dos fios, definição das rotas de alimentação (com o regulador LM2596) e o isolamento adequado dos circuitos de 24V com o shield OPS010. O controle do contraste do LCD, a escolha dos resistores ideais para os LEDs, além do roteamento seguro dos fios no interior das caixas, foram pontos que demandaram atenção especial e retrabalho.

Apesar dos desafios técnicos enfrentados ao longo do processo — incluindo instabilidades de comunicação, falhas nas conexões físicas e ajustes na lógica de controle — o sistema se mostrou funcional e estável após as fases finais de teste. Os dados térmicos são capturados com precisão e transmitidos de forma confiável ao banco de dados. Com base nesses dados, o segundo ESP32 realiza o acionamento automático dos atuadores de forma eficiente.

A interface local do display LCD permite o acompanhamento instantâneo da temperatura máxima, juntamente com os limites estabelecidos.

5. Orçamento

O quadro abaixo apresenta o detalhamento dos materiais utilizados no desenvolvimento do sistema ThermoTrack, com suas respectivas quantidades e valores de aquisição:

Tabela 1. Orçamento dos Materiais Utilizados.

Item	Descrição	Quantidade	Valor (R\$)
1	Sensor Térmico AMG8833 8x8	1	267,00
2	Display TFT 2.4" SPI Touch (ST7789V)	1	72,08
3	ESP32	2	94,00
4	Fonte Colmeia 24V 5A 120W	1	31,92
5	Jumpers (Macho-Fêmea + Macho-Macho)	2 Kits	32,98

6	Sinalizador BZ20-2L-R (Sirene)	1	19,60
7	Display LCD 16x2	1	16,00
8	Micro Ventilador 24V	1	24,00
9	Regulador de Tensão LM2596 (3A)	1	8,80
Total			R\$ 566,38

6. Comunicação entre Microcontroladores e Banco de Dados

O funcionamento do site ThermoTrack é baseado na integração entre linguagens de frontend e backend. Cada parte do sistema cumpre um papel fundamental, desde a coleta de dados até a exibição interativa no navegador. Abaixo, destacam-se os principais códigos que movimentam o projeto:

Envio de Dados pelo ESP32 para o Servidor Web

Este script PHP é o principal responsável por receber os dados enviados pelo ESP32 e inseri-los no banco de dados. Ele realiza:

- Validação da temperatura e do ID do dispositivo;
- Inserção na tabela leituras com o timestamp exato;
- Verificação dos limites definidos para acionar alertas automaticamente;
- Armazenamento do caminho da imagem térmica, quando presente.

Exemplo de uso:

```
if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $temperatura = $_POST["temperatura"];
    $dispositivo_id = $_POST["dispositivo_id"];
    // Valida e insere no banco
}
```

Figura 27. *inserirdados.php* responsável pelo recebimento de dados do sensor.

```
1 <?php
2 include("conexao.php");
3
4 // Garante que os dados venham como JSON (usado pelo ESP32)
5 $input = json_decode(file_get_contents("php://input"), true);
6
7 if ($_SERVER["REQUEST_METHOD"] === "POST" && $input) {
8     $dispositivo_id = isset($input["dispositivo_id"]) ? intval($input["dispositivo_id"]) : 1;
9     $temperatura_max = floatval($input["temperatura_max"]);
10    $imagem_base64 = $input["imagem_base64"];
11
12    // Salva leitura
13    $sql = "INSERT INTO leituras (dispositivo_id, temperatura) VALUES (?, ?)";
14    $stmt = $conn->prepare($sql);
15    $stmt->bind_param("id", $dispositivo_id, $temperatura_max);
16    $stmt->execute();
17    $leitura_id = $stmt->insert_id;
18    $stmt->close();
19
20    // Corrige e decodifica imagem base64
21    $imagem_base64 = preg_replace('#^data:image/\w+;base64,#i', '', $imagem_base64);
22    $imagem_base64 = str_replace(' ', '+', $imagem_base64); // corrige espaços
23    $imgData = base64_decode($imagem_base64);
24
25    if ($imgData) {
26        $imgPath = "imagens/display_" . $leitura_id . ".png";
27        file_put_contents($imgPath, $imgData);
28
29        // Atualiza caminho da imagem no banco
30        $sqlImg = "UPDATE leituras SET imagem_path = ? WHERE id = ?";
31        $stmtImg = $conn->prepare($sqlImg);
32        $stmtImg->bind_param("si", $imgPath, $leitura_id);
33        $stmtImg->execute();
34        $stmtImg->close();
35
36        // Também insere alerta com base na temperatura
37        $nivel = "médio";
38        if ($temperatura_max < 34.0) {
39            $nivel = "baixo";
40        } elseif ($temperatura_max > 39.0) {
41            $nivel = "crítico";
42        }
43        $mensagem = "Temp registrada: {$temperatura_max} °C";
44
45        $sqlAlert = "INSERT INTO alertas (dispositivo_id, mensagem, nivel, temperatura, timestamp) VALUES (?, ?, ?, ?, NOW())";
46        $stmtAlert = $conn->prepare($sqlAlert);
47        $stmtAlert->bind_param("issd", $dispositivo_id, $mensagem, $nivel, $temperatura_max);
48        $stmtAlert->execute();
49        $stmtAlert->close();
50
51        echo "OK";
52    } else {
53        echo "Erro ao decodificar imagem.";
54    }
55
56    $conn->close();
57 } else {
58     echo "Requisição inválida ou JSON malformado.";
59 }
60 }>
```

Fonte: Autoria própria.

Leitura e Atualização de Dados em Tempo Real

Esta é a página central de visualização em tempo real. Utiliza:

- HTML/CSS para estrutura e estilo moderno;
- JavaScript (fetch) para atualizar dinamicamente temperatura, status e gráficos sem recarregar;
- Chart.js para gerar os gráficos de linha com os dados de temperatura.

Destaque de código JavaScript:

```
setInterval(() => {
    fetch("statusatuais.php")
    .then(res => res.json())
    .then(data => {
        document.getElementById("temperaturaAtual").innerText = data.temperatura;
        // Atualiza gráfico, status, etc.
    });
}, 2000);
```


Figura 28. Página pgdashboards.php exibindo temperatura em tempo real.

```

22 $temperaturas = [];
23 $horarios = [];
24
25 if ($result->num_rows > 0) {
26     while ($row = $result->fetch_assoc()) {
27         $temperaturas[] = (float)$row["temperatura"];
28         $horarios[] = date("H:i", strtotime($row["timestamp"]));
29     }
30 }
31
32 // Dados do status atual + imagem
33 $sqlStatus = "SELECT temperatura, imagem_path FROM leituras ORDER BY timestamp DESC LIMIT 1";
34 $resultStatus = $conn->query($sqlStatus);
35
36 $temperaturaAtual = "--";
37 $status = "Indefinido";
38 $conformidade = "--";
39 $classeStatus = "text-secondary";
40 $classeConf = "text-secondary";
41 $caminhoImagem = "imagens/imagem_padrao.png"; // fallback
42
43 if ($resultStatus->num_rows > 0) {
44     $row = $resultStatus->fetch_assoc();
45     $temperaturaAtual = $row["temperatura"];
46     $caminhoImagem = $row["imagem_path"] ?: "imagens/imagem_padrao.png";
47
48     if ($temperaturaAtual < $limite_baixo) {
49         $status = "Abaixo do Ideal";
50         $conformidade = "Baixa";
51         $classeStatus = "text-warning";
52         $classeConf = "text-warning";
53     } elseif ($temperaturaAtual <= $limite_ideal) {
54         $status = "Dentro do Ideal";
55         $conformidade = "Alta";
56         $classeStatus = "text-success";
57         $classeConf = "text-primary";
58     } else {
59         $status = "Acima do Ideal";
60         $conformidade = "Crítica";
61         $classeStatus = "text-danger";
62         $classeConf = "text-danger";
63     }
64 }
65
66 $conn->close();
67 ?>
68
69 <!DOCTYPE html>
70 <html lang="pt-br">
71 <head>
72     <meta charset="UTF-8">
73     <meta name="viewport" content="width=device-width, initial-scale=1.0">
74     <title>Dashboard | ThermoTrack</title>
75     <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css">
76     <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap-icons/font/bootstrap-icons.css">
77     <style>
78         body { background-color: #f5f9ff; margin: 0; font-family: 'Segoe UI', sans-serif; }
79         .sidebar {
80             height: 100vh;
81             position: fixed;
82             top: 0; left: 0;
83             width: 260px;
84             background-color: #002b5c;
85             color: white;
86         }
87     </style>
88
89 </head>
90 <body>
91     <div class="d-flex">
92         <div class="sidebar">
93             <div class="p-3">
94                 <div class="text-center">
95                     <img alt="Logo ThermoTrack" class="img-fluid" style="max-width: 100px; height: auto;"/>
96                     <h2 style="margin: 0; font-size: 1.2em; font-weight: bold; color: white;">ThermoTrack
97                     <p style="margin: 0; font-size: 0.8em; color: white;">Monitoramento de Temperatura em Tempo Real
98                 </div>
99                 <div class="list-group mt-3" style="background-color: white; color: black; padding: 5px; border-radius: 5px;">
100                     <a href="#" class="list-group-item active">Dashboard</a>
101                     <a href="#" class="list-group-item">Configurações</a>
102                     <a href="#" class="list-group-item">Histórico</a>
103                     <a href="#" class="list-group-item">Sobre</a>
104                 </div>
105             </div>
106             <div class="flex-grow-1 p-3">
107                 <div class="text-center">
108                     <div style="font-size: 1.5em; font-weight: bold; color: #002b5c; margin-bottom: 10px;">Temperatura
```

Fonte: Autoria própria.

Este arquivo PHP consulta o banco de dados e retorna os dados mais recentes no formato JSON, utilizados pelos gráficos e cards da interface. Exemplo de resposta JSON:

```

{
  "temperatura": 78.5,
  "status": "Dentro do esperado",
  "conformidade": "OK"
}

```

7. Interface Web e Painel de Visualização

Arquitetura Frontend e Backend do Site

Define a identidade visual do site, incluindo:

- Gradientes de fundo azul e branco;
- Botões interativos, efeitos hover e responsividade;
- Estilo dos cards, containers e layout geral.

Exemplo de estilo visual:

Figura 29. API interna statusatuais.php retornando dados para o frontend.

```

1 k?php
2 $conexao = new mysqli("www.thyagoquintas.com.br", "engenharia_35", "araraazul", "engenharia_35");
3
4
5 // Última temperatura registrada
6 $sql = "SELECT temperatura FROM leituras ORDER BY id DESC LIMIT 1";
7 $res = $conexao->query($sql);
8 $temp = $res->fetch_assoc()['temperatura'] ?? 0;
9
10 // Limites
11 $sql = "SELECT limite_minimo, limite_maximo FROM limites WHERE id = 1";
12 $res = $conexao->query($sql);
13 $limites = $res->fetch_assoc();
14
15 echo json_encode([
16     "temperatura" => floatval($temp),
17     "limite_min" => floatval($limites['limite_minimo']),
18     "limite_max" => floatval($limites['limite_maximo'])
19 ]);
20 ?>
21

```

Fonte: Autoria própria.

```

.hero-section {
background: linear-gradient(to right, #ffffff, #007bff);
padding: 2rem;
border-radius: 1rem;
}

```

Figura 30. Trecho de código do style.css responsável pela estilização da interface.

```

22 $temperaturas = [];
23 $horarios = [];
24
25 if ($result->num_rows > 0) {
26     while ($row = $result->fetch_assoc()) {
27         $temperaturas[] = (float)$row["temperatura"];
28         $horarios[] = date("H:i", strtotime($row["timestamp"]));
29     }
30 }
31
32 // Dados do status atual + imagem
33 $sqlStatus = "SELECT temperatura, imagem_path FROM leituras ORDER BY timestamp DESC LIMIT 1";
34 $resultStatus = $conn->query($sqlStatus);
35
36 $temperaturaAtual = "--";
37 $status = "Indefinido";
38 $conformidade = "--";
39 $classeStatus = "text-secondary";
40 $classeConf = "text-secondary";
41 $caminhoImagem = "imagens/imagem_padrao.png"; // fallback
42
43 if ($resultStatus->num_rows > 0) {
44     $row = $resultStatus->fetch_assoc();
45     $temperaturaAtual = $row["temperatura"];
46     $caminhoImagem = $row["imagem_path"] ? "imagens/imagem_padrao.png";
47
48     if ($temperaturaAtual < $limite_baixo) {
49         $status = "Abaixo do Ideal";
50         $conformidade = "Baixa";
51         $classeStatus = "text-warning";
52         $classeConf = "text-warning";
53     } elseif ($temperaturaAtual <= $limite_ideal) {
54         $status = "Dentro do Ideal";
55         $conformidade = "Alta";
56         $classeStatus = "text-success";
57         $classeConf = "text-primary";
58     } else {
59         $status = "Acima do Ideal";
60         $conformidade = "Crítica";
61         $classeStatus = "text-danger";
62         $classeConf = "text-danger";
63     }
64 }
65
66 $conn->close();
67 ?>
68
69 <!DOCTYPE html>
70 <html lang="pt-br">
71 <head>
72     <meta charset="UTF-8">
73     <meta name="viewport" content="width=device-width, initial-scale=1.0">
74     <title>Dashboard | ThermoTrack</title>
75     <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css">
76     <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap-icons/font/bootstrap-icons.css">
77     <style>
78         body { background-color: #f5f9ff; margin: 0; font-family: 'Segoe UI', sans-serif; }
79         .sidebar {
80             height: 100vh;
81             position: fixed;
82             top: 0; left: 0;
83             width: 260px;
84             background-color: #002b5c;
85             color: white;
86         }

```

Fonte: Autoria própria.

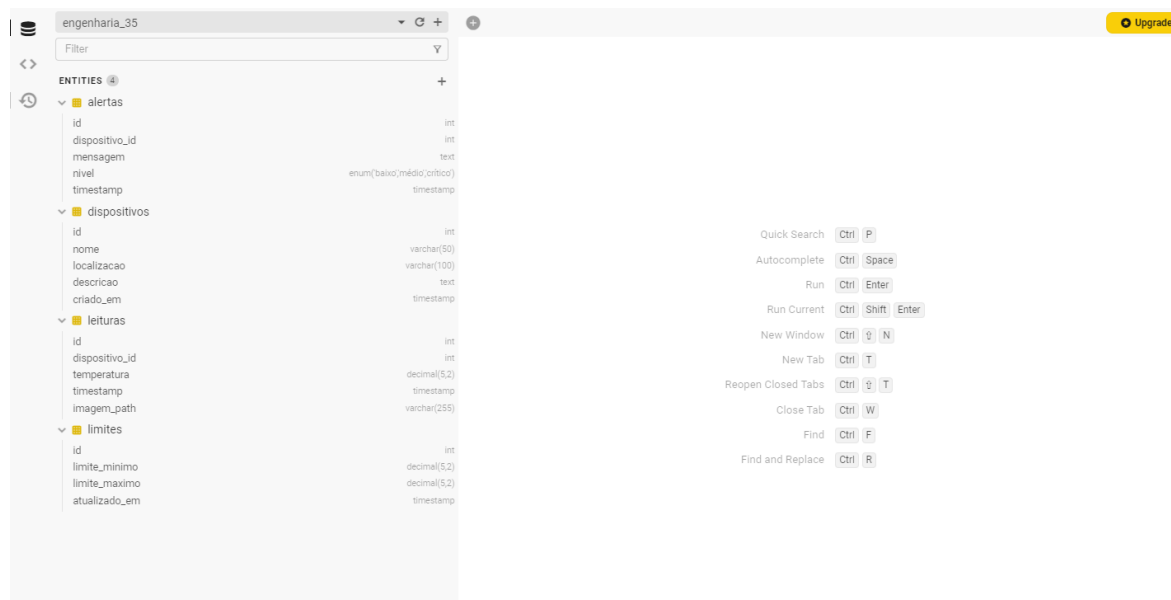
Banco de Dados MySQL

O ThermoTrack utiliza um banco de dados relacional MySQL, hospedado externamente, projetado para registrar e gerenciar dados em tempo real provenientes dos sensores conectados aos dispositivos. A modelagem do banco visa garantir integridade, rastreabilidade e desempenho na coleta e exibição das informações.

Quanto as entidades principais do Banco de Dados, é composto por quatro tabelas principais, cada uma com função específica:

- alertas: armazena registros gerados automaticamente quando a temperatura ultrapassa os limites aceitáveis.
- dispositivos: guarda informações dos sensores, como nome, localização e descrição técnica.
- leituras: registra os valores de temperatura capturados, o timestamp da leitura e, quando disponível, uma imagem térmica em base64.
- limites: define os valores mínimo e máximo considerados ideais, podendo ser atualizados conforme necessidade do usuário via interface web.

Figura 31. Banco de dados MySql no software Beekeeper.



Fonte: Autoria própria.

Fluxo de Entrada: Do Sensor ao Banco de Dados:

1. O Microcontrolador ESP32, conectado ao sensor térmico AMG8833, realiza leituras periódicas de temperatura.
2. Essas informações são transmitidas via requisição HTTP POST para o script inserirdados.php hospedado no servidor.
3. O script PHP valida os dados, insere na tabela leituras junto ao timestamp e ao ID do dispositivo.
4. Caso a temperatura estiver fora da faixa ideal (limites), um alerta é inserido automaticamente na tabela alertas.

Distribuição e Uso dos Dados na Interface Web:

- A página pgdashboards.php consulta os dados mais recentes para alimentar os gráficos e cards de status, exibindo em tempo real.
- O script statusatuais.php funciona como uma API interna, retornando dados em JSON para serem consumidos por JavaScript via fetch().

- Os alertas registrados podem ser filtrados por criticidade (baixo, médio, crítico) e apresentados na interface para rápida visualização.
- O usuário pode modificar os limites de temperatura diretamente pela interface; quando isso ocorre, um novo registro é salvo na tabela limites, com novo timestamp.

Esse fluxo integrado entre hardware, backend e banco de dados garante que o ThermoTrack opere com confiabilidade, permitindo respostas rápidas a variações térmicas e oferecendo dados claros para análise e tomada de decisão.

Interface Web e Dashboard

A interface web do ThermoTrack foi cuidadosamente projetada para oferecer uma experiência de uso responsiva, intuitiva e visualmente agradável, atendendo às necessidades tanto de usuários técnicos quanto operacionais. Desenvolvida com tecnologias modernas como HTML5, CSS3, JavaScript e o framework Bootstrap 5, a plataforma proporciona um ambiente funcional e adaptável.

Figura 32. Dashboard principal exibindo temperatura, status e conformidade em tempo real.



Fonte: Autoria própria.

A página principal apresenta um painel de controle centralizado, onde são exibidos em tempo real os dados mais recentes de temperatura obtidos por sensores térmicos. O layout adota um estilo limpo e moderno, com uma sidebar lateral que facilita a navegação entre as principais seções: Temperatura Atual, Status do Sistema, Conformidade, Gráficos e Alertas Recentes. Essa barra de navegação pode ser recolhida para maximizar o espaço útil da tela, otimizando a usabilidade em dispositivos variados, como tablets e notebooks.

A atualização dos dados ocorre de forma automática e contínua, por meio de requisições assíncronas utilizando a função `fetch()` em JavaScript puro. Esse mecanismo possibilita a atualização dos valores de temperatura, status e conformidade a cada dois segundos, sem a necessidade de recarregamento da página, garantindo fluidez e precisão na visualização das informações em tempo real.

Estrutura Tecnológica do Site

O site do ThermoTrack foi desenvolvido com base em quatro tecnologias essenciais da web: HTML, CSS, JavaScript e PHP. Cada uma delas desempenha um papel específico na estrutura, interatividade e integração do sistema. *HTML5 – Estrutura Base:*

A base da interface é construída com HTML5, que organiza todos os elementos visuais da página, tais como:

Títulos, seções e formulários: `<h2>`, `<form>`, `<div>`.

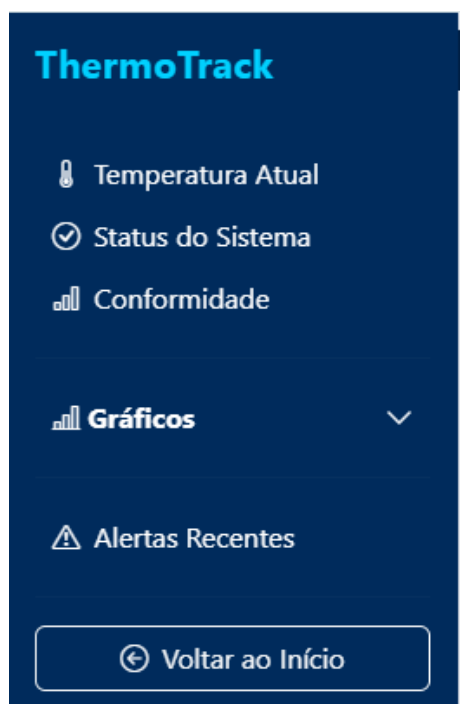
Áreas interativas: botões, `<canvas>` para gráficos, cartões de informação.

Elementos de navegação: menu lateral (sidebar) e janelas modais.

CSS3 + Bootstrap 5 – Estilo e Responsividade: A aparência visual é definida com CSS3, complementado pela biblioteca Bootstrap 5. Isso proporciona:

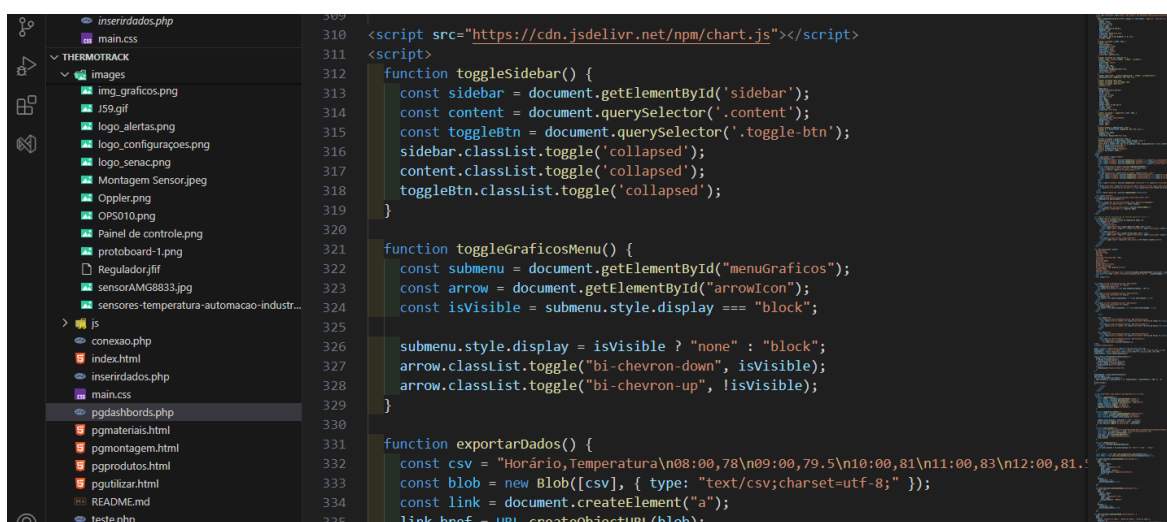
- Layout moderno e responsivo adaptado a diversos dispositivos;
- Estilização eficiente com classes utilitárias do Bootstrap;

Figura 33. Exemplo de menu lateral responsivo da interface web.



Fonte: Autoria própria.

Figura 34. Trecho de código com a função responsável pela atualização automática dos dados.



Fonte: Autoria própria.

- Personalizações específicas com CSS puro, como efeitos visuais, modais customizados e responsividade nos elementos gráficos;
- Inclusão de ícones funcionais por meio da biblioteca Bootstrap Icons.

O JavaScript permite uma interface dinâmica e responsiva, sem necessidade de recarregar a página. Entre os principais recursos implementados, destacam-se:

- Atualização automática dos dados (temperatura, status e conformidade) a cada dois segundos via fetch() com o endpoint statusatuais.php;
- Renderização em tempo real dos gráficos de linha e barras, por meio do endpoint graficosatuais.php;

Figura 35. Estilização com Bootstrap e CSS customizado aplicada ao layout.



Fonte: Autoria própria.

- Exibição condicional de elementos da página com base na interação do usuário;
- Visualização de imagem térmica codificada em base64, exibida em modais;
- Exportação dos dados para arquivos .csv diretamente pelo navegador.

O PHP é responsável pela camada lógica do servidor, realizando a ponte entre o frontend e o banco de dados MySQL. Suas principais funções são:

- Gerenciamento da conexão com o banco via script conexao.php;
- Consultas e inserções nas tabelas leituras e alertas;
- Processamento dos limites personalizados de temperatura definidos no formulário;
- Geração de endpoints JSON para consumo no frontend (graficosatuais.php, statusatuais.php);
- Exibição condicional de informações na página pgdashboards.php com manutenção de sessão.

A integração eficaz entre essas tecnologias resulta em um sistema robusto, com atualização contínua dos dados, interface fluida e adaptável a diferentes contextos operacionais.

8. Considerações Finais

O projeto desenvolvido demonstrou, na prática, a viabilidade de implementar soluções de monitoramento térmico inteligente utilizando tecnologias acessíveis, como o microcontrolador ESP32, sensores infravermelhos e shields industriais. A arquitetura proposta permitiu uma comunicação eficiente entre módulos, tomada de decisão autônoma com base em dados de temperatura, e exibição das informações em tempo real tanto localmente quanto por meio de uma interface web responsiva. A robustez e flexibilidade do sistema mostram que ele pode ser facilmente adaptado a diferentes ambientes industriais, com potencial real de aplicação em cenários de manutenção preditiva.

Além do desenvolvimento técnico, este trabalho envolveu uma ampla produção de documentação, incluindo este artigo acadêmico, um pitch de apresentação do produto, vídeo narrado explicativo, book digital e um banner descritivo do projeto, contribuindo para uma apresentação completa e profissional. O grupo participou também da **11ª Semana das Engenharias do Senac**, evento que proporcionou a oportunidade de apresentar o sistema a alunos, professores e convidados externos. A participação foi extremamente enriquecedora, não apenas pelo feedback recebido, mas também pela experiência de compartilhar nosso trabalho em um ambiente acadêmico e colaborativo.

Figura 36. Fotografia do dia de apresentação na 11ª Semana das Engenharias do Senac.



Fonte: Autoria própria.

Durante a exposição final, foi montada uma bancada com todos os elementos do sistema operando em tempo real, possibilitando aos avaliadores e visitantes observarem o funcionamento integrado do sensor, display, servidor e atuadores. Foram realizadas demonstrações de acionamento automático da ventoinha e sinalizador a partir de variações térmicas, reforçando a aplicabilidade da proposta.

Aproveitamos esta seção para registrar nossos agradecimentos. Em especial, ao professor **André Luiz**, que nos orientou de forma contínua nas aulas práticas de sábado, tirando dúvidas técnicas e contribuindo diretamente para o avanço do projeto. Também estendemos nosso reconhecimento ao professor **Marcello Marcelino Correia**, que coorientou a disciplina de *Projeto: Ciência e Tecnologia Aplicada II* e acompanhou nosso progresso desde o início. Por fim, agradecemos ao coordenador do curso, **Ricardo Luiz Ciuccio**, por sua presença na avaliação da apresentação final e por sempre incentivar os alunos a irem além do esperado.

Como perspectivas futuras, vislumbra-se a integração de algoritmos de inteligência artificial para análise preditiva com maior precisão, além da adição de sensores complementares, como umidade, vibração e pressão. Seria necessário também a melhora da caixa protetora do circuito, para melhor identidade visual. Esses aprimoramentos poderiam expandir a cobertura do sistema, tornando-o ainda mais completo, confiável e adaptável a contextos industriais complexos.

Referências

- SOUZA, D. *Automação Industrial com ESP32*. São Paulo: Novatec Editora, 2022.
- PANASONIC. *AMG8833 Infrared Array Sensor Grid-EYE Datasheet*. Panasonic Corporation.
- OPLLER. *Manual Técnico do Shield Industrial OPS010*. Disponível em: <https://loja.opller.com/produto/ops010>. Acesso em: 22 fev. 2025.
- ESPRESSIF SYSTEMS. *ESP32 Technical Reference Manual*. Disponível em: <https://www.espressif.com>. Acesso em: 1 mar. 2025.
- FAHAD, Shahzada. *AMG8833 Thermal Camera using ESP32 & TFT Display – Electronic Clinic*. Disponível em: <https://www.youtube.com/watch?v=uFoUAPQBCNY>. Acesso em: 15 mar. 2025.
- [1] SENAC. Modelo para artigo acadêmico. Disponível em: <https://www1.sp.senac.br/hotsites/blogs/revistainiciacao/>. Acesso em: 27 mai. 2025.
- [2] OKYSTAR. Sensor AMG8833 – Temperatura Infravermelha. Disponível em: <https://www.okystar.com/product-item/amg8833-temperature-sensor-module-okyn41204/>. Acesso em: 2 jun. 2025.
- [3] ELECTRONIC CLINIC. *AMG8833 + ESP32 + Display*. Disponível em: <https://www.youtube.com/watch?v=uFoUAPQBCNY>. Acesso em: 2 jun. 2025.
- [4] PIMORONI. *MLX90640 Thermal Camera Breakout*. Disponível em: <https://shop.pimoroni.com/products/mlx90640-thermal-camera-breakout>. Acesso em: 2 jun. 2025.
- [5] SMARTKITS. Sensor MLX90614. Disponível em: <https://www.smartkits.com.br/sensor-de-temperatura-ir-mlx90614-gy-906>. Acesso em: 2 jun. 2025.
- [6] SMARTKITS. Sensor DHT11 com LED. Disponível em: <https://www.smartkits.com.br/modulo-sensor-de-temperatura-e-umidade-dht11-com-led>. Acesso em: 2 jun. 2025.
- [7] VIDA DE SILÍCIO. *ESP32 ESP-WROOM-32 NodeMCU*. Disponível em: <https://vidadesilicio.com.br/produto/esp32-esp-wroom-32-nodemcu/>. Acesso em: 2 jun. 2025.
- [8] YOUTUBE. Tutorial de Display com Interpolação. Disponível em: <https://www.youtube.com/watch?v=zIwDnFDC0NE&t=813s>. Acesso em: 15 mar. 2025.
- [9] AMAZON. Display TFT ILI9341 240x320. Disponível em: <https://www.amazon.com.br/M%C3%B3dulo-240x320-Display-ILI9341-Arduino/dp/B07TZ3J127>. Acesso em: 22 mar. 2025.
- [10] OPLLER. Shield Industrial OPS010. Disponível em: <https://loja.opller.com/produto/ops010>. Acesso em: 22 fev. 2025.
- [11] USINA INFO. Display LCD 16x2. Disponível em: <https://www.usinainfo.com.br/display-arduino/display-lcd-16x2-com-fundo-azul-2304.html>. Acesso em: 6 jun. 2025.
- [12] SARTORIWEB. Sinalizador Sonoro BZ20-2L-R. Disponível em: <https://www.sartoriweb.com.br/p-sinalizador-sonoro-led-vm-220v-bz20-2l-r-i11235-25154>. Acesso em: 6 jun. 2025.
- [13] CASA DA ROBÓTICA. Micro Ventilador 24V. Disponível em: <https://www.casadarobotica.com/>. Acesso em: 6 jun. 2025.

[15] LOJA LUNICOM. Fonte 24V 3A MS-75-24. Disponível em: <https://www.lojalunicom.com.br/fonte-de-alimentao-bivolt-chaveada-ms-75-24-24v-3amp/prod-5339397/>. Acesso em: 6 jun. 2025.

[16] CASA DA ROBÓTICA. Protoboard 400 furos. Disponível em: <https://www.casadarobotica.com/prototipagem-e-ferramentas/prototipagem/protoboard/protoboard-400-furos-pontos>. Acesso em: 6 jun. 2025.